

```

1  # =====
2  # RELATÓRIO AUTOMÁTICO DE VENDAS
3  #
4  # Autor: Luciano
5  #
6  # Objetivo:
7  # Ler automaticamente todas as planilhas de vendas da pasta,
8  # consolidar os dados, tratar inconsistências, gerar resumos
9  # e criar um relatório profissional em Excel.
10 # =====
11
12
13 # =====
14 # IMPORTAÇÃO DAS BIBLIOTECAS
15 # =====
16
17 import pandas as pd
18 import glob
19
20 from openpyxl import load_workbook
21 from openpyxl.styles import Font, PatternFill, Alignment
22 from openpyxl.chart import BarChart, Reference
23 from openpyxl.utils import get_column_letter
24
25
26
27 # =====
28 # 1 - LOCALIZAR AUTOMATICAMENTE TODAS AS PLANILHAS
29 # =====
30
31 # Procura todos os arquivos iniciados por "vendas_"
32
33 arquivos = glob.glob("vendas_*.xlsx")
34
35
36
37 # Lista onde serão armazenados todos os DataFrames
38 lista_dfs = []
39
40
41
42 # =====
43 # 2 - LER TODAS AS PLANILHAS
44 # =====
45
46 for arquivo in arquivos:
47
48     # Lê o arquivo
49
50     df = pd.read_excel(arquivo)
51
52     # Cria uma coluna informando de qual arquivo o registro veio
53
54     df["Arquivo de Origem"] = arquivo
55
56     # Guarda o DataFrame
57
58     lista_dfs.append(df)
59
60
61
62 # =====
63 # 3 - JUNTAR TODOS OS DATAFRAMES
64 # =====
65
66 dados = pd.concat(lista_dfs, ignore_index=True)
67
68
69
70 # =====
71 # 4 - TRATAR DADOS AUSENTES
72 # =====
73

```

```

74 # As linhas que possuem alguma informação ausente serão
75 # enviadas para uma aba chamada Inconsistências.
76
77 inconsistencias = dados[dados.isnull().any(axis=1)]
78
79
80 # Remove registros incompletos da base principal
81
82 dados = dados.dropna()
83
84
85
86 # =====
87 # 5 - REMOVER DUPLICIDADES
88 # =====
89
90 dados = dados.drop_duplicates()
91
92
93
94 # =====
95 # 6 - GARANTIR O TIPO DOS DADOS
96 # =====
97
98 dados["Quantidade"] = pd.to_numeric(dados["Quantidade"])
99
100 dados["Valor Unitário"] = pd.to_numeric(dados["Valor Unitário"])
101
102
103
104 # =====
105 # 7 - CRIAR A COLUNA TOTAL
106 # =====
107
108 dados["Total"] = dados["Quantidade"] * dados["Valor Unitário"]
109
110
111
112 # =====
113 # 8 - GERAR RESUMO POR VENDEDOR
114 # =====
115
116 resumo_vendedores = (
117     dados
118     .groupby("Vendedor")
119     .agg(
120         Quantidade_de_Vendas=("Quantidade", "sum"),
121         Total_Vendido=("Total", "sum")
122     )
123     .reset_index()
124 )
125
126
127
128
129
130
131
132
133 # =====
134 # 9 - GERAR RESUMO POR PRODUTO
135 # =====
136
137 resumo_produtos = (
138     dados
139     .groupby("Produto")
140     .agg(
141         Quantidade_Vendida=("Quantidade", "sum"),
142         Faturamento=("Total", "sum")
143     )
144 )

```

```

147
148     .reset_index()
149
150 )
151
152
153
154 # =====
155 # 10 - EXPORTAR PARA O EXCEL
156 # =====
157
158 with pd.ExcelWriter("relatorio_final.xlsx",
159                     engine="openpyxl") as escritor:
160
161     dados.to_excel(
162         escritor,
163         sheet_name="Dados Consolidados",
164         index=False
165     )
166
167     resumo_vendedores.to_excel(
168         escritor,
169         sheet_name="Resumo por Vendedor",
170         index=False
171     )
172
173     resumo_produtos.to_excel(
174         escritor,
175         sheet_name="Resumo por Produto",
176         index=False
177     )
178
179     inconsistencias.to_excel(
180         escritor,
181         sheet_name="Inconsistências",
182         index=False
183     )
184
185
186
187 # =====
188 # 11 - ABRIR O ARQUIVO PARA FORMATAÇÃO
189 # =====
190
191 planilha = load_workbook("relatorio_final.xlsx")
192
193
194
195 # =====
196 # 12 - FORMATAÇÃO DAS ABAS
197 # =====
198
199 for aba in planilha.worksheets:
200
201     # Cabeçalho azul
202
203     for celula in aba[1]:
204
205         celula.fill = PatternFill(
206             fill_type="solid",
207             fgColor="1F4E78"
208         )
209
210         celula.font = Font(
211             color="FFFFFF",
212             bold=True
213         )
214
215         celula.alignment = Alignment(horizontal="center")
216
217     # Ajuste automático das colunas
218
219     for coluna in aba.columns:

```

```

220
221     maior = 0
222
223     letra = get_column_letter(coluna[0].column)
224
225     for celula in coluna:
226
227         if celula.value:
228
229             maior = max(maior, len(str(celula.value)))
230
231     aba.column_dimensions[letra].width = maior + 3
232
233     # =====
234     # 13 - FORMATAÇÃO DA DATA
235     # =====
236
237     aba = planilha["Dados Consolidados"]
238
239     # Procura a coluna chamada Data
240     coluna_data = None
241
242     for celula in aba[1]:
243         if celula.value == "Data":
244             coluna_data = celula.column
245             break
246
247     # Formata todas as datas
248     if coluna_data:
249         for linha in range(2, aba.max_row + 1):
250             aba.cell(row=linha,
251                     column=coluna_data).number_format = "dd/mm/yyyy"
252
253
254     # =====
255     # 14 - FORMATAÇÃO DE MOEDA
256     # =====
257
258     # =====
259     # FORMATAÇÃO DE MOEDA
260     # =====
261
262     campos_monetarios = {
263         "Dados Consolidados": ["Valor Unitário", "Total"],
264         "Resumo por Vendedor": ["Total_Vendido"],
265         "Resumo por Produto": ["Faturamento"]
266     }
267
268     for nome_aba, colunas in campos_monetarios.items():
269
270         aba = planilha[nome_aba]
271
272         # Localiza quais colunas devem receber formatação
273         mapa_colunas = {}
274
275         for celula in aba[1]:
276             if celula.value in colunas:
277                 mapa_colunas[celula.value] = celula.column
278
279         # Aplica formato monetário somente nas colunas encontradas
280         for coluna in mapa_colunas.values():
281
282             for linha in range(2, aba.max_row + 1):
283
284                 celula = aba.cell(row=linha, column=coluna)
285
286                 if isinstance(celula.value, (int, float)):
287                     celula.number_format = 'R$ #,##0.00'
288
289
290
291     # =====
292     # 15 - CRIAR O GRÁFICO

```

```

293 # =====
294
295 aba_resumo = planilha["Resumo por Vendedor"]
296
297 grafico = BarChart()
298
299 grafico.title = "Total vendido por vendedor"
300
301 grafico.y_axis.title = "Valor vendido (R$)"
302
303 grafico.x_axis.title = "Vendedor"
304
305 grafico.y_axis.majorGridlines = None
306
307 valores = Reference(
308     aba_resumo,
309     min_col=3,
310     min_row=1,
311     max_row=aba_resumo.max_row
312 )
313
314
315
316
317
318
319
320
321 nomes = Reference(
322     aba_resumo,
323     min_col=1,
324     min_row=2,
325     max_row=aba_resumo.max_row
326 )
327
328
329
330
331
332
333
334
335 grafico.add_data(
336     valores,
337     titles_from_data=True
338 )
339
340
341
342
343 grafico.set_categories(nomes)
344
345 grafico.height = 12
346
347 grafico.width = 20
348
349 aba_resumo.add_chart(grafico, "E2")
350
351
352
353 # =====
354 # 16 - SALVAR O ARQUIVO
355 # =====
356
357 planilha.save("relatorio_final.xlsx")
358
359
360
361 # =====
362 # 17 - RESPONDER ÀS PERGUNTAS
363 # =====
364
365 ranking = resumo_vendedores.sort_values(

```

```

366         by="Total_Vendido",
367         ascending=False
368     ).reset_index(drop=True)
369
370
371     primeiro = ranking.iloc[0]
372
373     segundo = ranking.iloc[1]
374
375
376     diferenca = primeiro["Total_Vendido"] - segundo["Total_Vendido"]
377
378
379     ticket_medio = (
380         primeiro["Total_Vendido"] /
381         primeiro["Quantidade_de_Vendas"]
382     )
383
384
385
386     print("=" * 50)
387
388     print("RELATÓRIO CONCLUÍDO COM SUCESSO!")
389
390     print("=" * 50)
391
392     print()
393
394     print(f"Maior vendedor: {primeiro['Vendedor']}")
395
396     print(f"Total vendido: R$ {primeiro['Total_Vendido']:.2f}")
397
398     print()
399
400     print(f"Diferença para o segundo colocado: R$ {diferenca:.2f}")
401
402     print()
403
404     print(
405         f"Quantidade vendida: "
406         f"{primeiro['Quantidade_de_Vendas']}"
407     )
408
409     print(
410         f"Ticket médio: "
411         f"R$ {ticket_medio:.2f}"
412     )
413
414     print()
415
416     print(
417         "Para identificar se o maior resultado veio de "
418         "mais vendas ou de vendas mais caras, compare o "
419         "ticket médio do vencedor com o dos demais vendedores."
420     )
421
422     print()
423
424     print("Arquivo gerado: relatorio_final.xlsx")

```